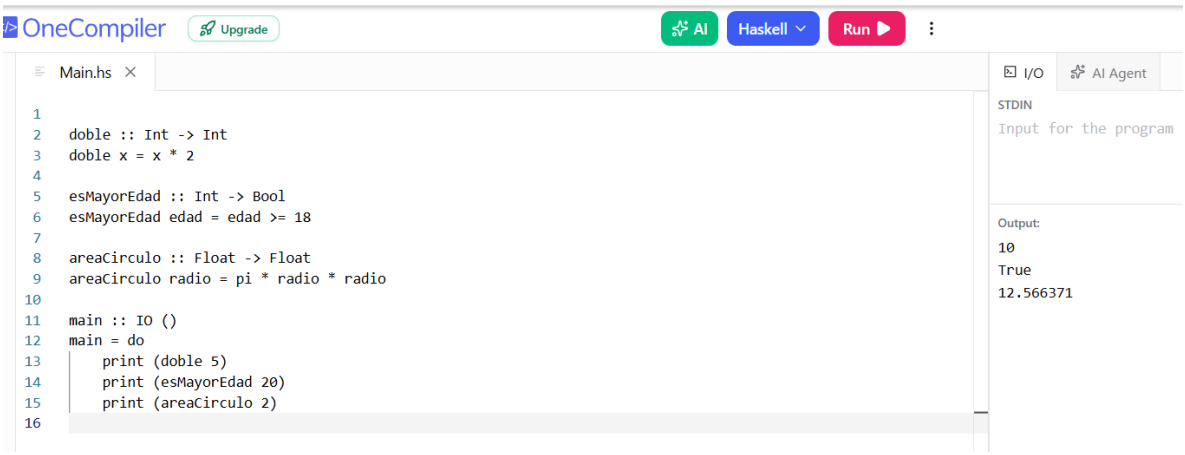
Laboratorio 01 – Haskell

Ejercicio 1




```haskell
doble :: Int -> Int
doble x = x * 2

esMayorEdad :: Int -> Bool
esMayorEdad edad = edad >= 18

areaCirculo :: Float -> Float
areaCirculo radio = pi * radio * radio

main :: IO ()
main = do
    print (doble 5)
    print (esMayorEdad 20)
    print (areaCirculo 2)
```

¿Qué significa que Haskell sea un lenguaje fuertemente tipado?

Significa que los valores y las funciones tienen tipos definidos y que el compilador verifica que se utilicen de manera compatible. Esto ayuda a detectar errores cuando se intenta utilizar un tipo de dato incorrecto.

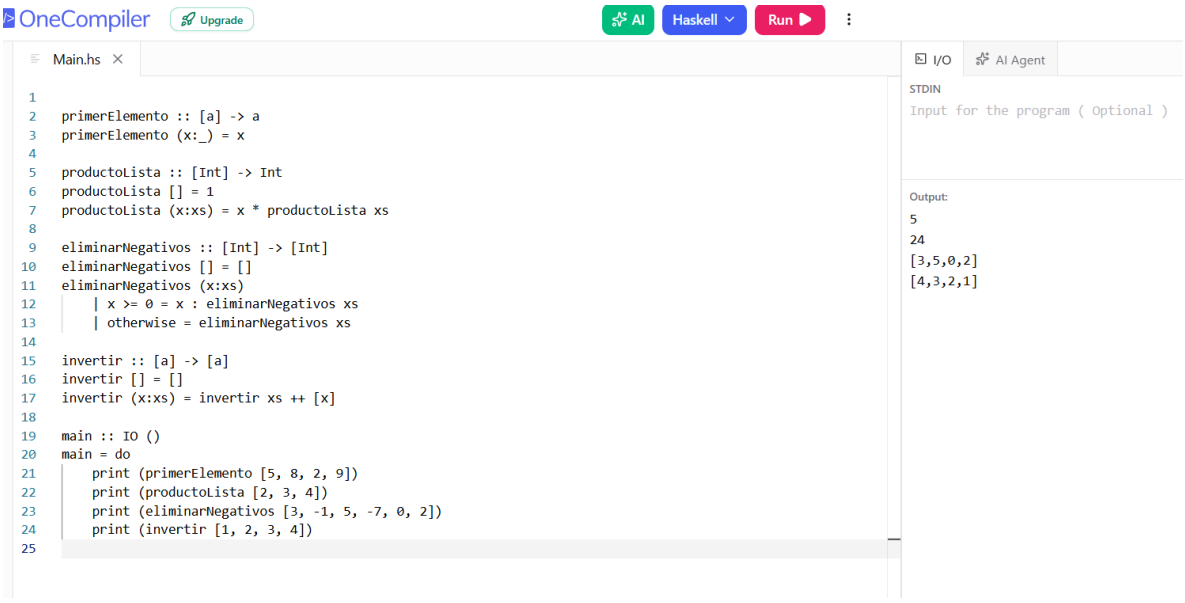
¿Qué es la evaluación perezosa (*lazy evaluation*)?

Es una característica de Haskell que consiste en evaluar una expresión solamente cuando su resultado es necesario. Esto permite trabajar con listas infinitas calculando únicamente los elementos que se necesitan.

¿Cuál es la diferencia entre Int y Float?

Int representa números enteros, como 5, -3 y 20. Float representa números que pueden tener una parte decimal, como 3.5 y 2.75. En este ejercicio se utiliza Float para calcular el área de un círculo, porque el resultado puede contener decimales.

Ejercicio 2




```haskell

primerElemento :: [a] -> a
primerElemento (x:_) = x

productoLista :: [Int] -> Int
productoLista [] = 1
productoLista (x:xs) = x * productoLista xs

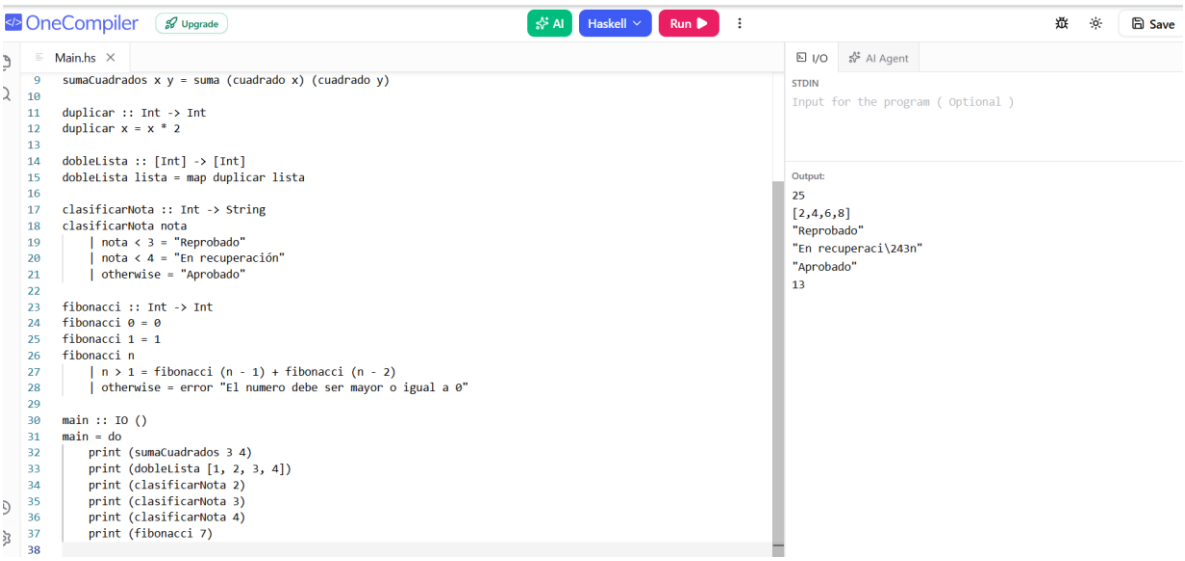
eliminarNegativos :: [Int] -> [Int]
eliminarNegativos [] = []
eliminarNegativos (x:xs)
    | x >= 0 = x : eliminarNegativos xs
    | otherwise = eliminarNegativos xs

invertir :: [a] -> [a]
invertir [] = []
invertir (x:xs) = invertir xs ++ [x]

main :: IO ()
main = do
    print (primerElemento [5, 8, 2, 9])
    print (productoLista [2, 3, 4])
    print (eliminarNegativos [3, -1, 5, -7, 0, 2])
    print (invertir [1, 2, 3, 4])

```

Ejercicio 3




```haskell
sumaCuadrados x y = suma (cuadrado x) (cuadrado y)

duplicar :: Int -> Int
duplicar x = x * 2

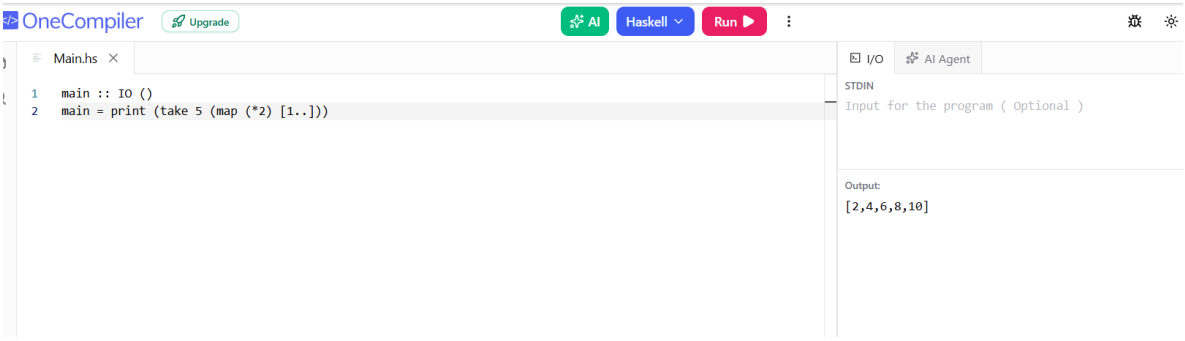
dobleLista :: [Int] -> [Int]
dobleLista lista = map duplicar lista

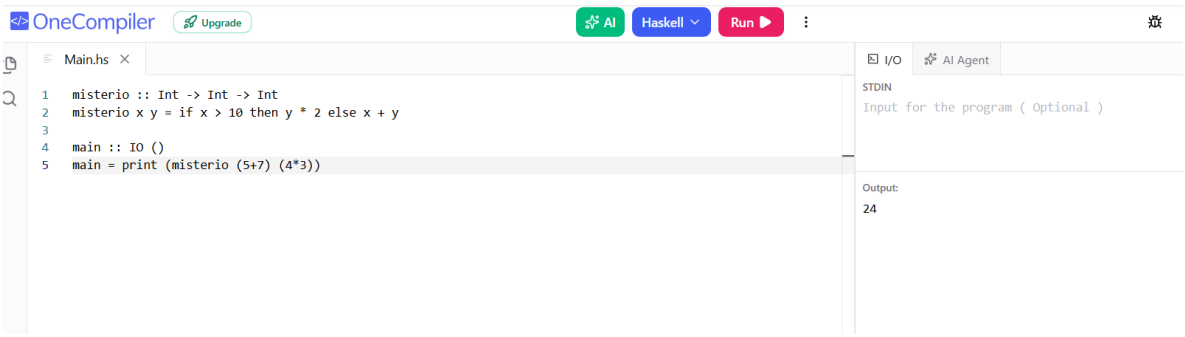
clasificarNota :: Int -> String
clasificarNota nota
    | nota < 3 = "Reprobado"
    | nota < 4 = "En recuperación"
    | otherwise = "Aprobado"

fibonacci :: Int -> Int
fibonacci 0 = 0
fibonacci 1 = 1
fibonacci n
    | n > 1 = fibonacci (n - 1) + fibonacci (n - 2)
    | otherwise = error "El numero debe ser mayor o igual a 0"

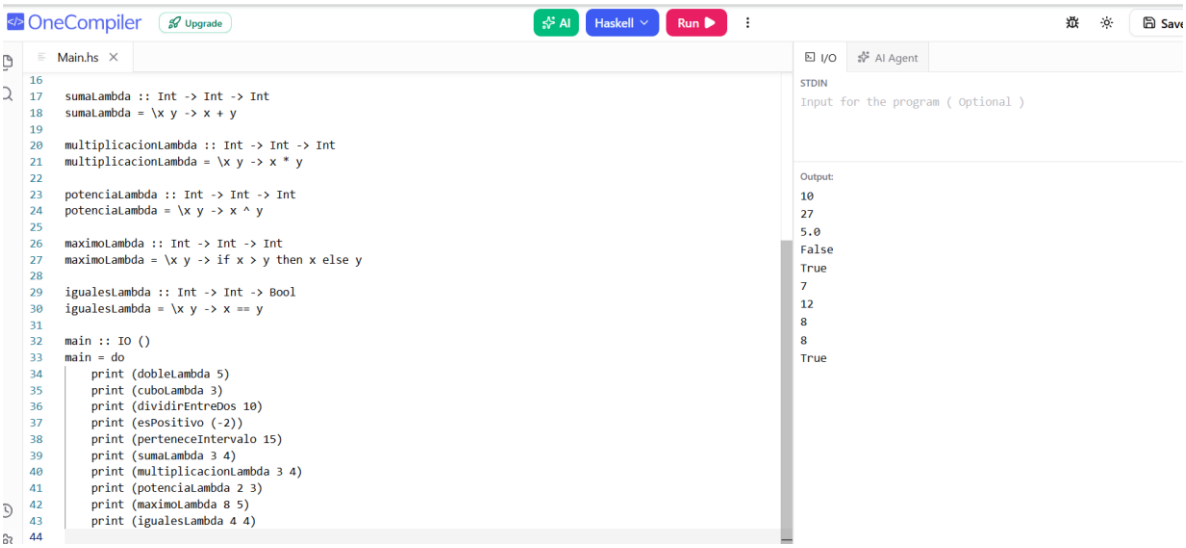
main :: IO ()
main = do
    print (sumaCuadrados 3 4)
    print (dobleLista [1, 2, 3, 4])
    print (clasificarNota 2)
    print (clasificarNota 3)
    print (clasificarNota 4)
    print (fibonacci 7)
```

Ejercicio 4





Ejercicio 5



Retrospectiva

Durante este laboratorio practiqué la definición de funciones en Haskell y la especificación de sus tipos de datos. También trabajé con listas, funciones de orden superior como map, evaluación perezosa, recursión y funciones lambda.

Aprendí que la evaluación perezosa permite trabajar con listas infinitas calculando solamente los elementos necesarios. Además, comprendí cómo la recursión permite resolver problemas como el producto de una lista y la sucesión de Fibonacci.

Este laboratorio me ayudó a comprender mejor los fundamentos de la programación funcional y la importancia de definir correctamente las funciones y sus tipos.